College Code: 9209

N.S.N College of Engineering and Technology

Department of Computer Science and Engineering

PROJECT DOCUMENTATION

WHATNEXT VISION MOTORS

A Salesforce-Based Parcel Management System

Team Members

S.No	Name	Role
1	MAYILSAMY S	Team Lead
2	JEEVANRAJA K	Team Member
3	MOHAMAD ANSHA I	Team Member
4	MOHANDASS V	Team Member

Team ID: SWTID -2026-9488

ABSTRACT

WhatNext Vision Motors is a Salesforce-based vehicle and dealership management application developed to replace manual registers and spreadsheet-based tracking with a centralised cloud platform. The application is built on the Salesforce Lightning platform and uses six custom objects: Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive and Vehicle Service Request. A custom Lightning app provides navigation, and record-triggered flows and Apex code provide automation.

An Apex trigger and handler class prevent a vehicle order from being saved when the selected vehicle has no stock, and reduce stock when an order is saved with the status Confirmed. A batch Apex class, scheduled daily at 12:00 PM, confirms pending orders when stock is available. A record-triggered flow retrieves the customer's address and looks up a vehicle dealer whose location exactly matches it, and a second flow sends an e-mail to the customer for a scheduled test drive.

This report documents the design and implementation as evidenced by the project's source code and Salesforce screenshots. It records which functions were verified, namely the out-of-stock validation, the test-drive e-mail and the scheduled job, and which were not, and it identifies limitations such as the possibility of duplicate stock deduction, exact-match rather than distance-based dealer selection, and the absence of verified reports, dashboards, security configuration and AI functionality. Improvements are proposed as future enhancements.

Keywords: Salesforce, Lightning, Apex trigger, Batch Apex, Record-Triggered Flow, Vehicle Order Management, Dealer Management.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 1
1.1 Introduction 1
1.2 Background 1
1.3 Problem Statement 1
1.4 Existing System 2
1.5 Limitations of the Existing System 2
1.6 Proposed System 2
1.7 Project Objectives 3
1.8 Scope of the Project 3
CHAPTER 2: SYSTEM ANALYSIS AND REQUIREMENTS 4
2.1 Functional Requirements 4
2.2 Non-Functional Requirements 4
2.3 Hardware Requirements 5
2.4 Software Requirements 5
2.5 Technologies Used 6
2.6 Salesforce Platform Overview 6
2.7 System Architecture 6
2.8 Salesforce Architecture 7
2.9 User Interface Overview 7
2.10 System Workflow 7
CHAPTER 3: SYSTEM DESIGN AND IMPLEMENTATION 9
3.1 Development Environment 9
3.2 Salesforce Custom Objects and Data Model 9
3.3 Custom Fields and Relationships 10
3.4 Tabs and Lightning App 13
3.5 Application Modules 14
3.6 Vehicle Ordering and Stock Management 15
3.7 Dealer Selection 17
3.8 Test Drive Reminder Flow 20
3.9 Salesforce Apex Automation 22
3.10 Salesforce Security 25
3.11 Agentforce and AI Functionality 26
3.12 Reports and Dashboards 26
3.13 Implementation Limitations and Concerns 26
CHAPTER 4: TESTING AND RESULTS 28
4.1 Testing Approach 28
4.2 Test Cases 28

4.3 Results and Observations 29
4.4 Summary of Implementation Status 30
CHAPTER 5: CONCLUSION AND FUTURE ENHANCEMENTS 32
5.1 Project Summary 32
5.2 Objectives Achieved 32
5.3 Advantages 32
5.4 Current Limitations 33
5.5 Future Enhancements 33
5.6 Conclusion 34
References 34

LIST OF FIGURES

Figure 1: Salesforce Setup Home Page of the Developer Organisation 9
Figure 2: New Custom Object Creation Form 9
Figure 3: Creating the Vehicle Model Picklist Field 12
Figure 4: Creating the Stock Quantity Number Field 13
Figure 5: Defining the Dealer Lookup Field on the Vehicle Object 13
Figure 6: Creating a Custom Object Tab for Vehicle 13
Figure 7: New Lightning App: App Details and Branding 13
Figure 8: Assigning the System Administrator Profile to the Lightning App 14
Figure 9: VehicleOrderTrigger Apex Trigger 15
Figure 10: VehicleOrderTriggerHandler Class in the Developer Console 17
Figure 11: Get Customer Information Flow Element 18
Figure 12: Get Nearest Dealer Flow Element 18
Figure 13: Assign Dealer to Order Update Records Element 19
Figure 14: Auto Assign Dealer Flow Canvas Before Activation 19
Figure 15: Scheduled Path Configuration for the Test Drive Reminder 21
Figure 16: Send Email Action Configuration 21
Figure 17: Test Drive Reminder Flow Canvas (Active) 21
Figure 18: Test Drive Reminder E-mail Received 22
Figure 19: VehicleOrderBatch Class in the Developer Console 23
Figure 20: VehicleOrderBatchScheduler Class 24
Figure 21: Scheduling the Batch Job in the Execute Anonymous Window 24
Figure 22: Scheduled Jobs Page Showing Daily Vehicle Order Processing 25
Figure 23: Vehicle Order Form Showing the Out-of-Stock Validation Error 30

LIST OF TABLES

Table 1: Functional Requirements 4
Table 2: Non-Functional Requirements 4
Table 3: Recommended Client Hardware 5
Table 4: Software Requirements 5
Table 5: Technology Stack 6
Table 6: Workflow Stages and Implementing Mechanism 7
Table 7: Custom Objects and Relationships 10
Table 8: Fields of the Vehicle Object 10
Table 9: Fields of the Vehicle Dealer Object 11
Table 10: Fields of the Vehicle Customer Object 11
Table 11: Fields of the Vehicle Order Object 11
Table 12: Fields of the Vehicle Test Drive Object 12
Table 13: Fields of the Vehicle Service Request Object 12
Table 14: Configuration of the Auto Assign Dealer Flow 17
Table 15: Configuration of the Test Drive Reminder Flow 20
Table 16: Execution Sequence for a Vehicle Order 25
Table 17: Functional Test Cases 28
Table 18: Implementation Status Summary 30
Table 19: Objectives and Status 32

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRM	Customer Relationship Management
CSE	Computer Science and Engineering
DML	Data Manipulation Language
FR	Functional Requirement
ID	Identifier
NFR	Non-Functional Requirement
SOQL	Salesforce Object Query Language
UI	User Interface

CHAPTER 1: INTRODUCTION

1.1 Introduction

WhatNext Vision Motors is a Salesforce-based vehicle and dealership management system. It brings vehicle information, stock quantity, customer records, vehicle orders, dealer information, test drives and service requests into a single cloud application built on the Salesforce Lightning platform. Business rules that were previously handled by hand, such as checking whether a vehicle is available before accepting an order, are implemented as Apex code and Salesforce Flows.

This report documents the design and implementation of the application. It describes the custom objects and fields, the Lightning app, the record-triggered flows, the Apex trigger, handler, batch class and scheduler, and the testing carried out. The report is based on the project documentation and the Salesforce screenshots and source code supplied by the project team. Where a feature is described in the project documentation but is not supported by screenshots or test evidence, the report says so.

1.2 Background

Vehicle dealerships and manufacturers typically manage orders, stock and customer enquiries through a combination of registers, spreadsheets and separate software tools. When these records are kept apart, stock figures become outdated, orders are accepted for vehicles that are no longer available, and customers are not told about the progress of their requests. Customer Relationship Management (CRM) platforms address this by keeping all related records in one database and allowing business rules to be automated.

Salesforce is a cloud CRM platform that provides a configurable data model (objects, fields and relationships), a declarative automation tool (Flow), a programming language (Apex) and a user interface framework (Lightning Experience). The WhatNext Vision Motors project uses these capabilities to model the vehicle ordering process of a dealership network.

1.3 Problem Statement

The project documentation identifies two groups affected by the existing process. An operations manager who manages vehicle orders finds that stock information is maintained manually and that systems are disconnected, which causes stress and a concern about wrong orders and stock errors. A customer who wants to place a vehicle

order finds that availability is unclear and that updates are delayed, which leaves the customer uncertain.

The problem addressed by this project is therefore the absence of a single, automated system that validates stock at the time of ordering, keeps stock figures consistent with confirmed orders, links customers to dealers, and reminds customers about test drives.

1.4 Existing System

In the existing arrangement described in the project documentation, vehicle orders and stock are recorded in manual registers and spreadsheets. Dealer and customer information is held in separate files. Operations staff follow up manually with dealers and customers, and compile status information by hand.

1.5 Limitations of the Existing System

- Stock information is entered and updated manually and can be out of date.
- Orders can be accepted for vehicles that are out of stock because no automatic check exists.
- Records are scattered across multiple files and systems, so there is no single view of vehicles, customers, dealers and orders.
- Order status and test-drive information must be communicated through manual follow-ups, causing delays and cancellations.
- Reporting depends on collating data from several sources.

1.6 Proposed System

The proposed system is a Salesforce Lightning application named **WhatNext Vision Motors**. Its main elements are as follows.

- Six custom objects (Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive and Vehicle Service Request) that store the business data and relate to one another through lookup fields.
- An Apex trigger and handler class that block an order from being saved when the selected vehicle has no stock, and reduce stock by one when an order is saved with the status Confirmed.

- A record-triggered flow, Auto Assign Dealer, that runs when a Vehicle Order is created with status Pending, retrieves the customer's information and looks up the Vehicle Dealer whose location equals the customer's address.
- A record-triggered flow, Test Drive Reminder, that sends an e-mail to the customer for a test drive whose status is Scheduled.
- A batch Apex class, VehicleOrderBatch, and a scheduler class, VehicleOrderBatchScheduler, scheduled to run daily at 12:00 PM to confirm pending orders when stock is available.

The behaviour of each element, together with its verified status and limitations, is described in Chapters 3 and 4.

1.7 Project Objectives

- To store vehicle, dealer, customer, order, test-drive and service-request data in a centralised Salesforce data model.
- To prevent vehicle orders for vehicles that are out of stock.
- To keep vehicle stock quantity consistent with confirmed orders through automation.
- To associate a customer's order with a dealer based on the customer's location using Salesforce Flow.
- To remind customers of scheduled test drives by e-mail.
- To process pending orders automatically using batch Apex that runs on a daily schedule.

1.8 Scope of the Project

The scope of the project covers vehicle and stock information, customer information, dealer information, vehicle orders, test drives and service requests, together with the automation associated with vehicle ordering and test-drive reminders. The project was developed and tested in a Salesforce Developer Edition organisation.

The following items are outside the verified scope of this report because the supplied project evidence does not demonstrate them: reports and dashboards, role-based security configuration beyond the assignment of the System Administrator profile to the app, distance-based dealer calculation, and any Agentforce or Einstein AI functionality. They are discussed as limitations or future enhancements.

CHAPTER 2: SYSTEM ANALYSIS AND REQUIREMENTS

2.1 Functional Requirements

The functional requirements of the system, as defined in the project documentation, are listed in Table 1. The final column records whether the supplied evidence demonstrates the requirement.

Table 1: Functional Requirements

FR No	Requirement	Evidence in project files
FR-1	Vehicle management	Vehicle object and fields defined; Vehicle Model and Stock Quantity field creation shown
FR-2	Dealer management	Vehicle Dealer object and fields defined; no record screenshots
FR-3	Customer order creation	Vehicle Order form shown with Vehicle Customer, Vehicle, Order Date and Status fields
FR-4	Stock validation	Apex trigger and handler; out-of-stock error message shown
FR-5	Dealer auto-assignment	Flow configured (exact location match); execution result not shown
FR-6	Test drive scheduling and reminder	Test Drive object and reminder flow defined; reminder e-mail received
FR-7	Reports and dashboards	Listed in project documentation; no report or dashboard evidence supplied

2.2 Non-Functional Requirements

Table 2: Non-Functional Requirements

Requirement	Description	How it is addressed
Usability	Easy navigation	Salesforce Lightning Experience with a custom Lightning app and object tabs
Security	Controlled access to data	Salesforce authentication; the app is assigned to the System Administrator profile. Detailed access controls are not verified
Reliability	Accurate automation	Stock check in a before-save trigger; limitations identified in Section 3.12

Requirement	Description	How it is addressed
Performance	Fast order processing	Trigger logic is bulkified using collections; batch processing uses a batch size of 50
Availability	24/7 cloud access	Provided by the Salesforce cloud platform
Scalability	Multi-dealer support	Dealers are stored as records; orders can relate to many vehicles and customers

2.3 Hardware Requirements

Because the application runs in the Salesforce cloud, only a client device with a web browser is required. The configuration in Table 3 is recommended for development and use.

Table 3: Recommended Client Hardware

Component	Recommended specification
Processor	Dual-core processor, 2 GHz or higher
Memory	4 GB RAM or more
Storage	1 GB free disk space for browser cache and downloaded files
Display	1366 x 768 resolution or higher
Network	Stable broadband Internet connection

2.4 Software Requirements

Table 4: Software Requirements

Software	Description
Salesforce Developer Edition org	Development and deployment platform for objects, flows and Apex
Salesforce Lightning Experience	User interface used to manage records and build the Lightning app
Setup, Object Manager and Flow Builder	Declarative configuration tools
Developer Console	Used to write and save Apex classes and triggers (API version 62.0 is shown in the screenshots)
Web browser	Current version of Google Chrome, Microsoft Edge or Mozilla Firefox

Software	Description
Operating system	Any operating system that supports the above browsers

2.5 Technologies Used

Table 5: Technology Stack

Layer	Technology
User interface	Salesforce Lightning Experience; custom Lightning app; custom object tabs
Data storage	Salesforce custom objects, fields and lookup relationships
Declarative automation	Record-triggered Flows (Flow Builder), including a scheduled path and a Send Email action
Programmatic automation	Apex trigger and handler class
Background processing	Batch Apex (Database.Batchable) and Scheduled Apex (Schedulable)
Reporting	Salesforce Reports and Dashboards (planned; not verified)

The project documentation also lists "Validation Rules" among the automation technologies. No validation rule configuration was supplied; stock validation is implemented in the Apex trigger.

2.6 Salesforce Platform Overview

Salesforce is a multi-tenant cloud platform. Data is stored in **objects**, which are similar to database tables; each object has **fields** and may be linked to other objects through **relationships**. A **lookup relationship** is a loosely coupled link in which a field on one object stores the identifier of a record of another object. **Salesforce Flow** is a declarative tool for automating business processes, and a **record-triggered flow** starts when a record is created, updated or deleted. **Apex** is a strongly typed, Java-like language that runs on the Salesforce servers; an **Apex trigger** executes before or after data manipulation events, **Batch Apex** processes large numbers of records in manageable chunks, and **Scheduled Apex** runs a class at a specified time using a cron expression. **Lightning Experience** is the user interface through which users work with records, and **Lightning Apps** group tabs into a navigable application.

2.7 System Architecture

The system follows the standard layered architecture of a Salesforce application.

- **Presentation layer:** the WhatNext Vision Motors Lightning app with tabs for each custom object.
- **Business logic layer:** record-triggered flows for dealer lookup and test-drive reminders, the VehicleOrderTrigger and VehicleOrderTriggerHandler for stock validation and stock update, and the VehicleOrderBatch class for processing pending orders.
- **Data layer:** six custom objects linked by lookup relationships.
- **Scheduling layer:** VehicleOrderBatchScheduler, registered as the scheduled job Daily Vehicle Order Processing.

2.8 Salesforce Architecture

Within Salesforce, the application is configured entirely inside a single organisation. Records are saved through the Lightning UI; each save passes through the Salesforce order of execution, in which before-save triggers run first, then the record is saved, then after-save triggers and record-triggered flows run. This sequence explains the design of the application: the stock check is placed in a **before** trigger so that an invalid order is rejected before it is saved, while the stock update is placed in an **after** trigger so that it occurs only once the order is stored. Scheduled jobs run independently of user actions, using the Apex scheduler.

2.9 User Interface Overview

The Lightning app is named **WhatNext Vision Motors**. According to the project documentation, its navigation items include the custom object tabs (Vehicle, Dealer, Customer, Order, Test Drive and Service Request) together with Reports and Dashboard, and it is made available to the System Administrator profile. Users create and edit records through standard Lightning record forms; an example of the Vehicle Order form is shown in Figure 23. A screenshot of the final app home page was not supplied, so this report does not reproduce one.

2.10 System Workflow

The intended workflow follows the customer journey defined in the project documentation: Inquiry, Vehicle Selection, Order Placement, Dealer Assignment, Stock Validation, Test Drive, Confirmation and Reporting. Table 6 maps each stage to the implemented mechanism.

Table 6: Workflow Stages and Implementing Mechanism

Stage	Implementation
Vehicle selection	Vehicle records with model, price and stock quantity; the order references a Vehicle
Order placement	A Vehicle Order record is created with customer, vehicle, order date and status (Pending, Confirmed, Delivered, Canceled)
Stock validation	Before insert/update trigger rejects orders for vehicles whose stock quantity is zero or less
Dealer assignment	Auto Assign Dealer flow looks up a Vehicle Dealer whose location equals the customer's address
Test drive	Vehicle Test Drive record; Test Drive Reminder flow sends an e-mail to the customer
Confirmation	After-save handler reduces stock when status is Confirmed; batch job confirms pending orders when stock exists
Reporting	Planned; not verified

CHAPTER 3: SYSTEM DESIGN AND IMPLEMENTATION

3.1 Development Environment

The application was built in a Salesforce Developer Edition organisation. The developer account is created from the Salesforce developer sign-up page and activated through an e-mail verification link, after which the user is redirected to the Setup home page. Figure 1 shows the Setup home page of the organisation used for the project.

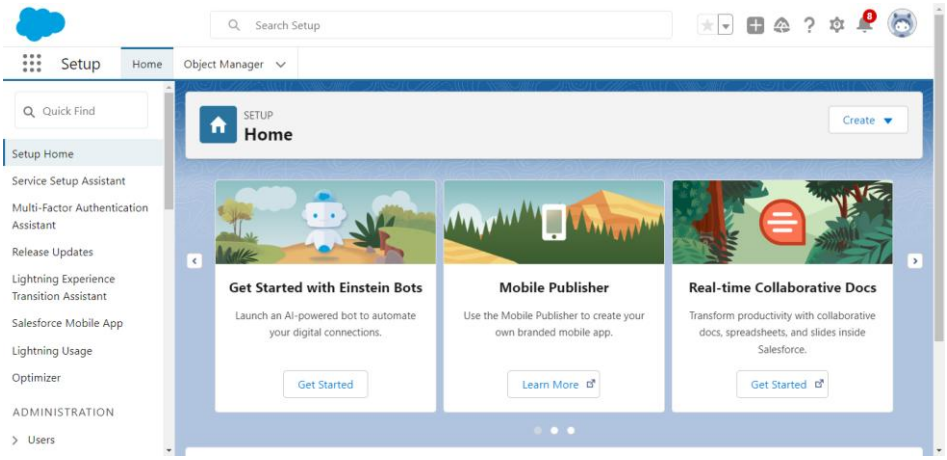


Figure 1: Salesforce Setup Home Page of the Developer Organisation

3.2 Salesforce Custom Objects and Data Model

Six custom objects were designed to model the business. Each object is created from Setup > Object Manager > Create > Custom Object by entering a label, plural label and record name, and by enabling Allow Reports and Allow Search. Figure 2 shows the New Custom Object form, and Table 7 lists the objects.

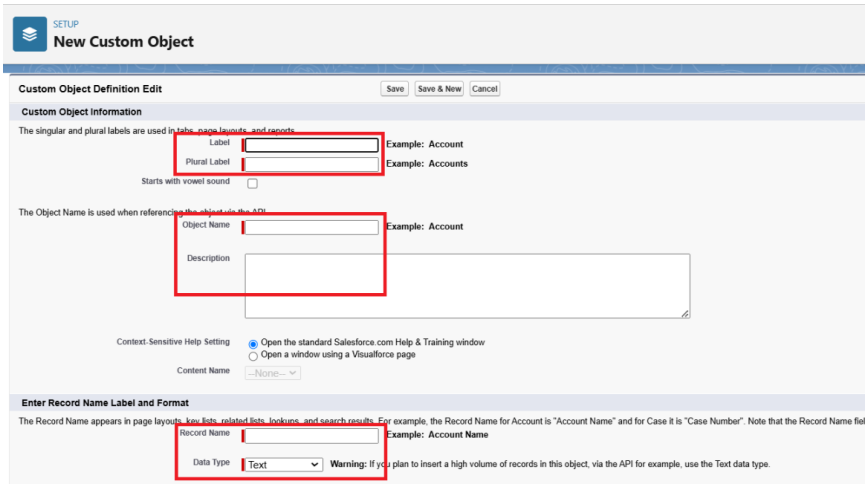


Figure 2: New Custom Object Creation Form

Table 7: Custom Objects and Relationships

Object (API name)	Purpose	Relationships (as specified)
Vehicle (Vehicle__c)	Stores vehicle details and stock	Related to Dealer and Orders
Vehicle Dealer (Vehicle_Dealer__c)	Stores authorised dealer information	Related to Orders
Vehicle Customer (Vehicle_Customer__c)	Stores customer details	Related to Orders and Test Drives
Vehicle Order (Vehicle_Order__c)	Tracks vehicle purchases	Related to Customer and Vehicle
Vehicle Test Drive (Vehicle_Test_Drive__c)	Tracks test drive bookings	Related to Customer and Vehicle
Vehicle Service Request (Vehicle_Service_Request__c)	Tracks vehicle servicing requests	Related to Customer and Vehicle

The Vehicle__c object appears in Object Manager as a custom object, confirming its creation. The remaining objects follow the same creation procedure as described in the project documentation, and their API names are taken from the Apex code and flow configuration where referenced (Vehicle__c, Vehicle_Order__c, Vehicle_Customer__c, Vehicle_Dealer__c and Vehicle_Test_Drive__c).

3.3 Custom Fields and Relationships

The fields below are those specified in the project documentation. The screenshots in this section show the creation of the Vehicle Model picklist, the Stock Quantity number field and the lookup from Vehicle to Vehicle Dealer. The remaining fields are documented as specified and are not individually evidenced by screenshots.

3.3.1 Vehicle

Table 8: Fields of the Vehicle Object

Field API name	Data type	Description
Vehicle Name (record name)	Text	Name of the vehicle; the specification lists Vehicle_Name__c
Vehicle_Model__c	Picklist	Sedan, SUV, EV, etc.
Stock_Quantity__c	Number	Number of units in stock; used by the trigger and batch class
Price__c	Currency	Price of the vehicle

Field API name	Data type	Description
Dealer__c	Lookup (Vehicle Dealer)	Dealer that holds the vehicle
Status__c	Picklist	Available, Out of Stock, Discontinued

3.3.2 Vehicle Dealer

Table 9: Fields of the Vehicle Dealer Object

Field API name	Data type	Description
Dealer_Name__c	Text	Name of the dealer (record name)
Dealer_Location__c	Text	Location of the dealer; compared with the customer address by the flow
Dealer_Code__c	Auto Number	System-generated dealer code
Phone__c	Phone	Contact number
Email__c	Email	Contact e-mail

3.3.3 Vehicle Customer

Table 10: Fields of the Vehicle Customer Object

Field API name	Data type	Description
Customer_Name__c	Text	Name of the customer (record name)
Email__c	Email	Customer e-mail; recipient of the test-drive reminder
Phone__c	Phone	Contact number
Address__c	Text	Customer address; used by the dealer-lookup flow
Preferred_Vehicle_Type__c	Picklist	Sedan, SUV, EV, etc.

3.3.4 Vehicle Order

Table 11: Fields of the Vehicle Order Object

Field API name	Data type	Description
Vehicle_Customer__c	Lookup (Vehicle Customer)	Customer placing the order. The specification names this field Customer__c, but the flow references Vehicle_Customer__c
Vehicle__c	Lookup (Vehicle)	Vehicle that is ordered
Order_Date__c	Date	Date of the order

Field API name	Data type	Description
Status__c	Picklist	Pending, Confirmed, Delivered, Canceled

The Vehicle Order form in Figure 23 displays the record name VC-0003 together with the fields Vehicle Customer, Vehicle, Order Date and Status. The specification does not list a Dealer lookup field on Vehicle Order; this has a consequence for dealer assignment, discussed in Section 3.7.

3.3.5 Vehicle Test Drive

Table 12: Fields of the Vehicle Test Drive Object

Field API name	Data type	Description
Customer__c	Lookup (Vehicle Customer)	Customer who books the test drive
Vehicle__c	Lookup (Vehicle)	Vehicle for the test drive
Test_Drive_Date__c	Date	Date of the test drive; time source of the reminder flow
Status__c	Picklist	Scheduled, Completed, Canceled

3.3.6 Vehicle Service Request

Table 13: Fields of the Vehicle Service Request Object

Field API name	Data type	Description
Customer__c	Lookup (Vehicle Customer)	Customer requesting service
Vehicle__c	Lookup (Vehicle)	Vehicle to be serviced
Service_Date__c	Date	Requested service date
Issue_Description__c	Text	Description of the issue
Status__c	Picklist	Requested, In Progress, Completed

3.3.7 Field creation screenshots

A field is created from Object Manager > object > Fields & Relationships > New. For a picklist, the label and the list of values are entered (Figure 3); for a number field, the label and length are entered (Figure 4); and for a relationship, the type Lookup Relationship is selected and the related object is chosen (Figure 5).

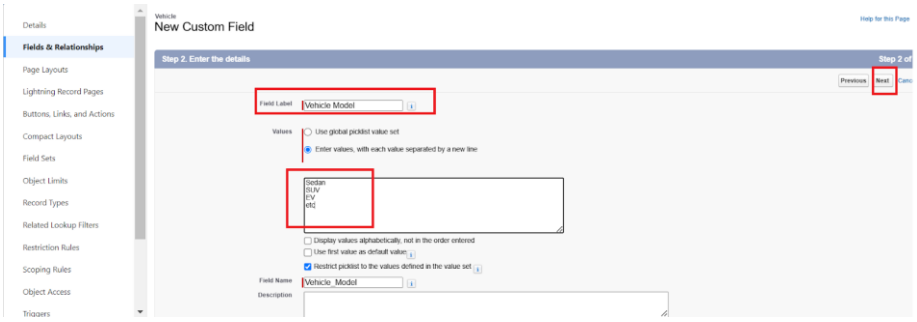


Figure 3: Creating the Vehicle Model Picklist Field

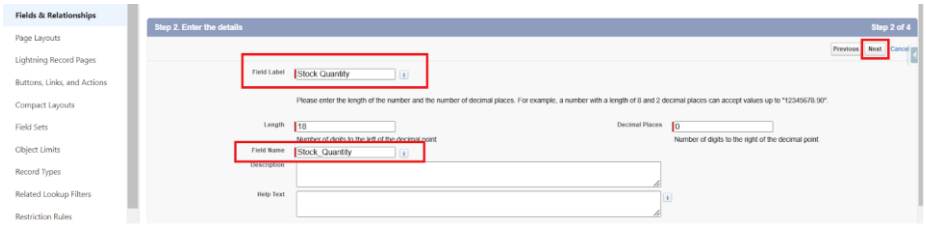


Figure 4: Creating the Stock Quantity Number Field

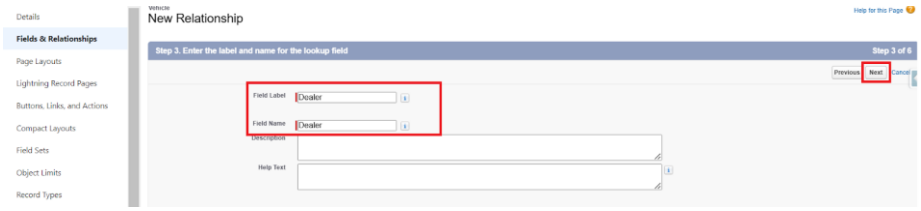


Figure 5: Defining the Dealer Lookup Field on the Vehicle Object

3.4 Tabs and Lightning App

A custom tab is created for each object from Setup > Tabs > Custom Object Tabs > New (Figure 6). The Lightning app is then created from Setup > App Manager > New Lightning App. The app name is WhatNext Vision Motors, the developer name is auto-populated, and a description is entered (Figure 7). Navigation items are chosen from the available items, and the System Administrator profile is moved to the Selected Profiles list so that it can access the app (Figure 8).

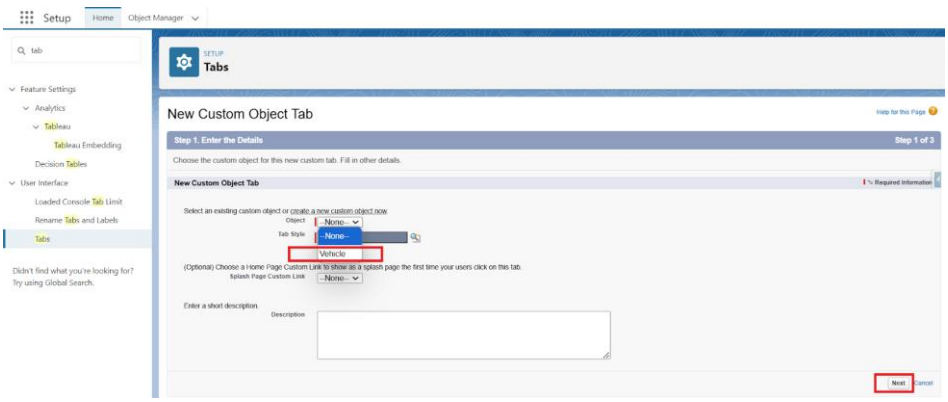


Figure 6: Creating a Custom Object Tab for Vehicle

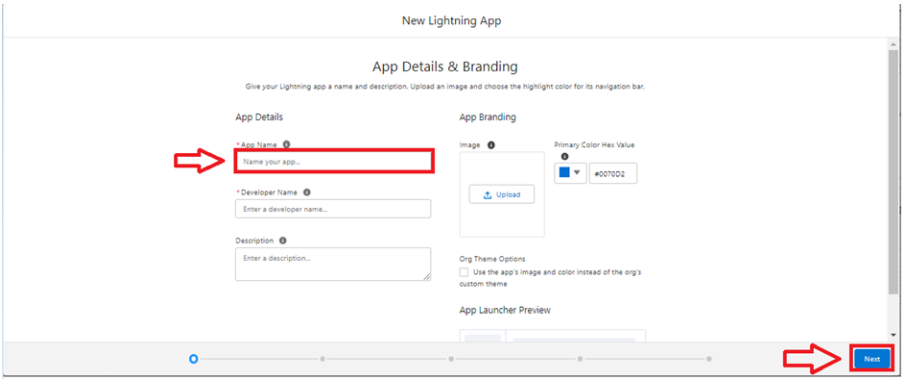


Figure 7: New Lightning App: App Details and Branding

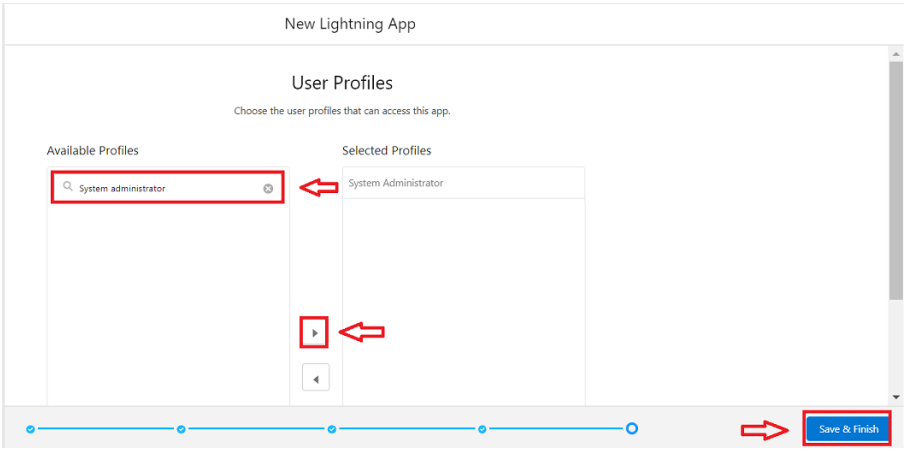


Figure 8: Assigning the System Administrator Profile to the Lightning App

The navigation-items screenshot in the source document shows items from a different organisation (for example, HandsMen Products), so it is not reproduced here.

3.5 Application Modules

3.5.1 Vehicle Management

This module maintains vehicle records, including the model, price, stock quantity, status and the dealer that holds the vehicle. The stock quantity is the central value used by the ordering logic: it is read by the before-save trigger to reject orders and is reduced by the after-save handler and the batch class.

3.5.2 Customer Management

This module maintains customer records with name, e-mail, phone, address and preferred vehicle type. The e-mail address is used as the recipient of test-drive reminders and the address is used by the dealer-lookup flow.

3.5.3 Vehicle Dealer Management

This module maintains authorised dealer records with name, location, an auto-numbered dealer code, phone and e-mail. The location text is compared with the customer address by the flow.

3.5.4 Vehicle Order Management

This module records each order with the customer, vehicle, order date and status. Order creation is subject to the stock validation trigger and the Auto Assign Dealer flow. The status values are Pending, Confirmed, Delivered and Canceled.

3.5.5 Test Drive Management

This module records test-drive bookings with customer, vehicle, date and status (Scheduled, Completed, Canceled), and is supported by the Test Drive Reminder flow.

3.5.6 Service Request Management

This module records service requests with customer, vehicle, service date, issue description and status (Requested, In Progress, Completed). No automation for service requests was supplied.

3.6 Vehicle Ordering and Stock Management

Vehicle ordering and stock control are implemented in the Apex trigger VehicleOrderTrigger and its handler class VehicleOrderTriggerHandler. The trigger is defined on Vehicle_Order__c for the events before insert, before update, after insert and after update, and passes the context variables to the handler (Figure 9 and Listing 1).

```
trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after
        insert, after update) {
    VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap,
            Trigger.isBefore,
        Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);
}
```

Listing 1: VehicleOrderTrigger

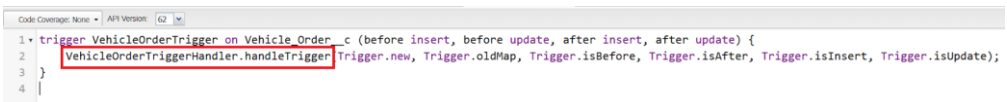


Figure 9: VehicleOrderTrigger Apex Trigger

3.6.1 Stock availability validation

In the before-save context, the handler method preventOrderIfOutOfStock collects the vehicle identifiers of the orders being saved, queries Stock_Quantity__c for those

vehicles in a single SOQL query, and calls addError on any order whose vehicle has a stock quantity of zero or less. The message shown to the user is "This vehicle is out of stock. Order cannot be placed." This validation applies to every insert and update of an order regardless of its status.

3.6.2 Stock deduction

In the after-save context, the handler method updateStockOnOrderPlacement selects orders whose Status__c equals Confirmed, retrieves the corresponding vehicles, and subtracts one from Stock_Quantity__c for each such order when the stock quantity is greater than zero. The changed vehicle records are saved with a single update statement. Stock deduction therefore occurs when an order is saved with the status Confirmed; orders with other statuses do not change stock.

3.6.3 Handler source code

```
public class VehicleOrderTriggerHandler {

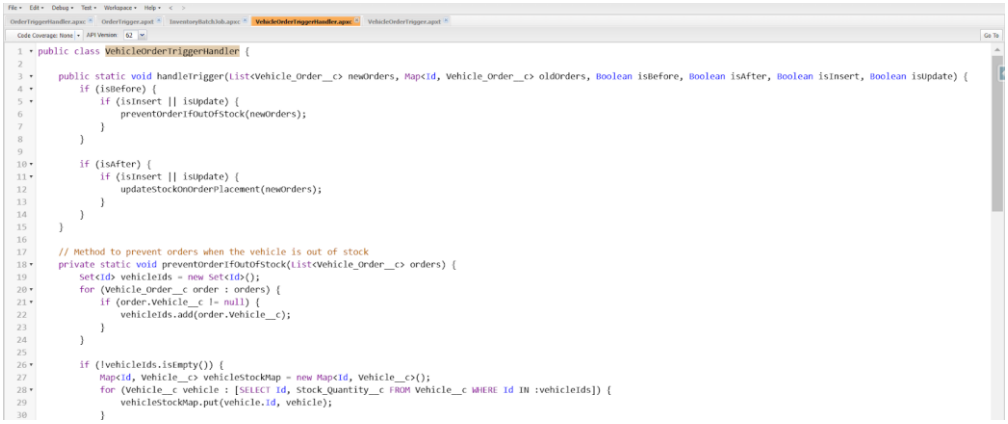
    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id,
            Vehicle_Order__c> oldOrders,
            Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
        if (isBefore) {
            if (isInsert || isUpdate) { preventOrderIfOutOfStock(newOrders); }
        }
        if (isAfter) {
            if (isInsert || isUpdate) { updateStockOnOrderPlacement(newOrders); }
        }
    }

    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) { vehicleIds.add(order.Vehicle__c); }
        }
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
                                       WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c <= 0) {
                        order.addError('This vehicle is out of stock. Order cannot be
                                placed.');
                    }
                }
            }
        }
    }

    // Method to update vehicle stock when an order is placed
    private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
                vehicleIds.add(order.Vehicle__c);
            }
        }
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
                                       WHERE Id IN :vehicleIds]) {
```

```
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        vehicle.Stock_Quantity__c -= 1;
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }
            if (!vehiclesToUpdate.isEmpty()) { update vehiclesToUpdate; }
        }
    }
}
```

Listing 2: VehicleOrderTriggerHandler (reformatted for readability; logic unchanged)



```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }

    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
```

Figure 10: VehicleOrderTriggerHandler Class in the Developer Console

3.6.4 Error handling for unavailable vehicles

When a user saves an order for a vehicle whose stock quantity is zero, Salesforce displays the message "We hit a snag" with the error "This vehicle is out of stock. Order cannot be placed." and the record is not saved. This behaviour was observed for order VC-0003 and is shown in Figure 23 in Chapter 4.

3.7 Dealer Selection

Dealer selection is implemented by the record-triggered flow **Auto Assign Dealer** (API name Auto_Assign_Dealer). It is created from Setup > Flows > New Flow > Start From Scratch > Record-Triggered Flow.

Table 14: Configuration of the Auto Assign Dealer Flow

Element	Configuration
Trigger	Object Vehicle Order; runs when a record is created; entry condition (all conditions met): Status__c Equals Pending

Element	Configuration
Get Customer Information (Get Records)	Object Vehicle Customer; filter Id Equals {!$Record.Vehicle_Customer__c}
Get Nearest Dealer (Get Records)	Object Vehicle Dealer; filter Dealer_Location__c Equals {!Get_Customer_Information.Address__c}
Assign Dealer to Order (Update Records)	Updates records using the IDs and field values from the record or record collection {!Get_Nearest_Dealer}

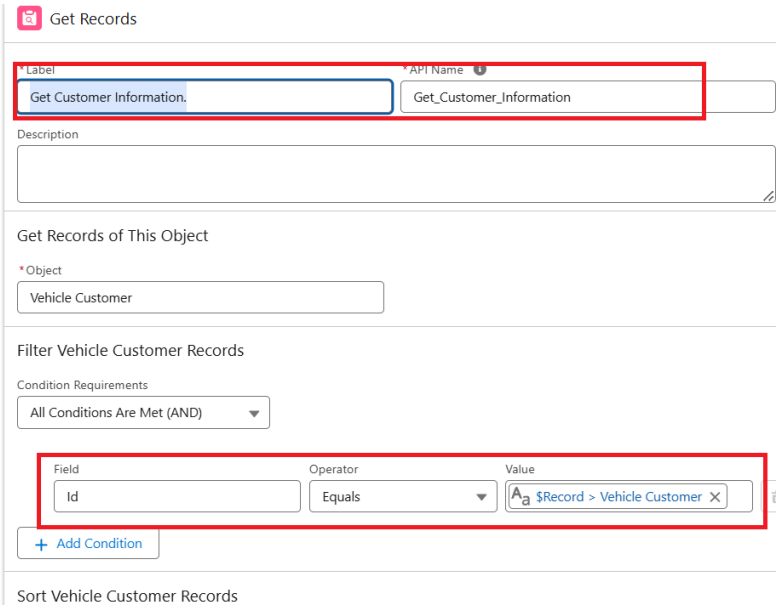


Figure 11: Get Customer Information Flow Element

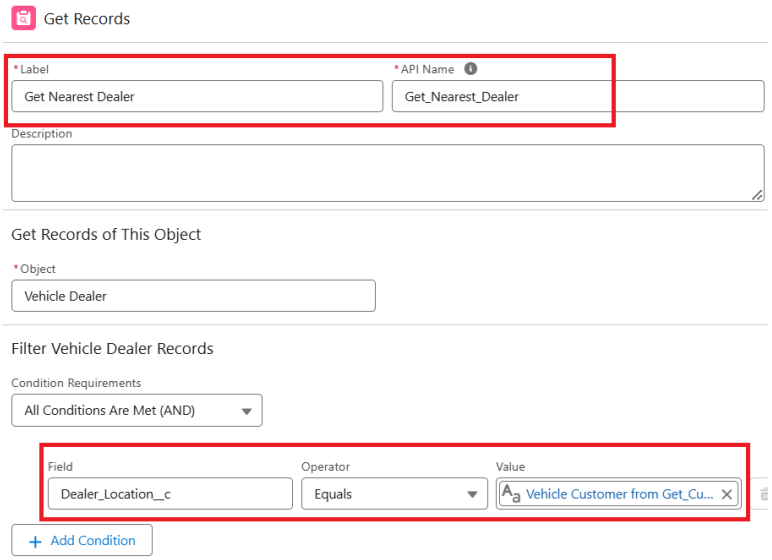


Figure 12: Get Nearest Dealer Flow Element

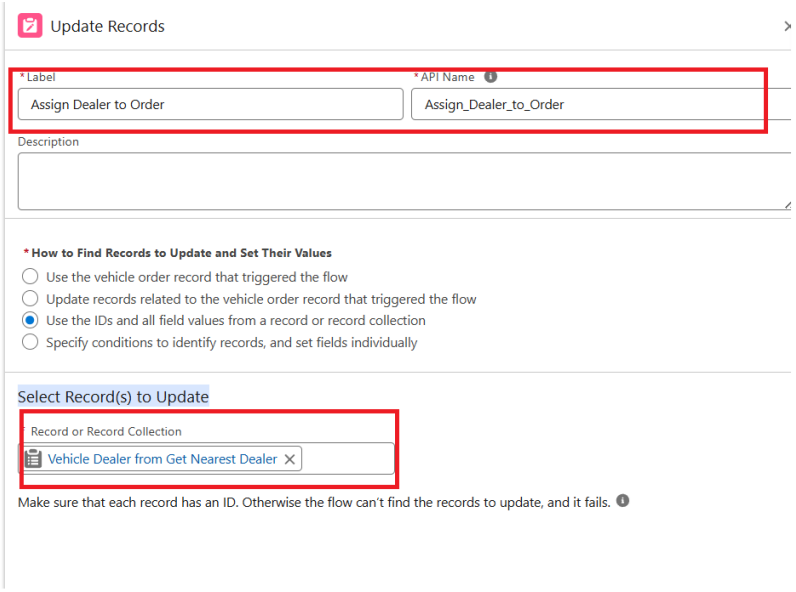


Figure 13: Assign Dealer to Order Update Records Element

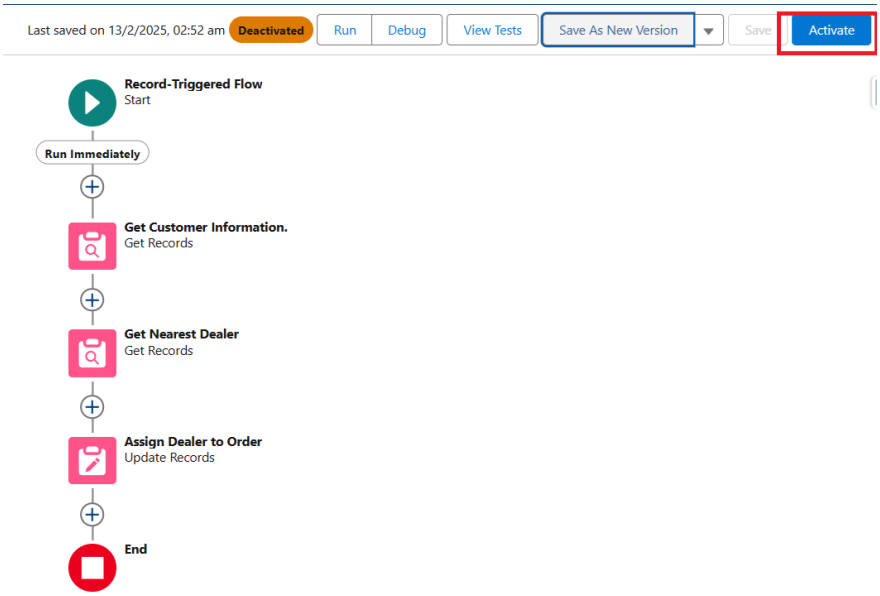


Figure 14: Auto Assign Dealer Flow Canvas Before Activation

3.7.1 Exact location matching versus nearest-dealer calculation

Although the element is labelled "Get Nearest Dealer", it does not calculate distance. It retrieves a Vehicle Dealer whose Dealer_Location__c text is **exactly equal** to the customer's Address__c text. A dealer is found only when the two text values match; no geolocation field, distance function or ranking is used anywhere in the supplied implementation. A genuine nearest-dealer calculation would require geolocation data for customers and dealers and a distance comparison, and is listed as a future enhancement.

3.7.2 Observations on the implementation

- The Update Records element updates the Vehicle Dealer record that was retrieved. The Vehicle Order specification contains no Dealer lookup field, and the flow does not set any field on the order, so the supplied evidence does not show a dealer being stored on the order.
- Figure 14 shows the flow canvas at the point of activation, when its status was Deactivated. No screenshot of an executed flow interview or of a resulting assignment was supplied; the function is therefore documented as configured but not verified.

3.8 Test Drive Reminder Flow

The record-triggered flow **Test Drive Reminder** (API name Test_Drive_Reminder) sends an e-mail to the customer for a test drive. It is defined as follows.

Table 15: Configuration of the Test Drive Reminder Flow

Element	Configuration
Trigger	Object Vehicle Test Drive; runs when a record is created or updated; entry condition: Status__c Equals Scheduled
Scheduled path	Label Reminder Before Test Drive; time source Test_Drive_Date__c; offset 1 Day(s) Before
Get Customer Information (Get Records)	Object Vehicle_Customer__c; filter Id Equals {!$Record.Customer__c}; first record only; all fields stored automatically
Send Test Drive Reminder (Action: Send Email)	Recipient {!Get_Customer_Information.Email__c}; subject "Reminder: Your Test Drive is Tomorrow!"; rich-text body from the text variable EmailSent; use line breaks enabled

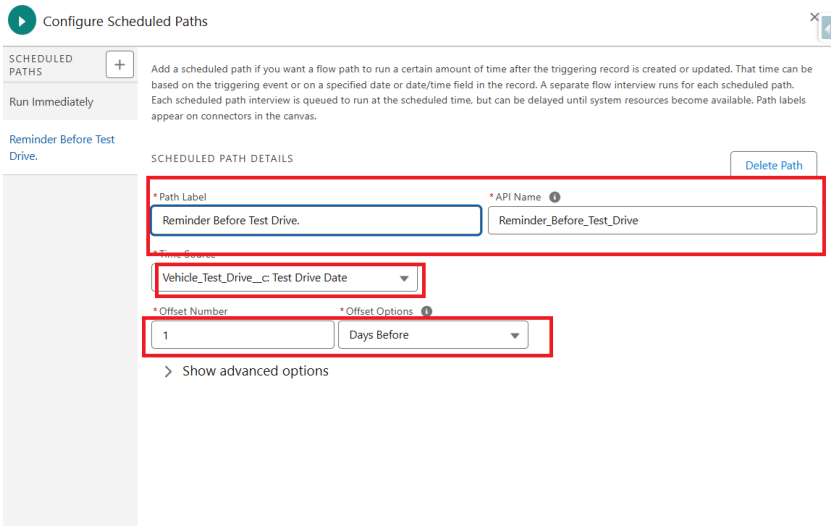


Figure 15: Scheduled Path Configuration for the Test Drive Reminder

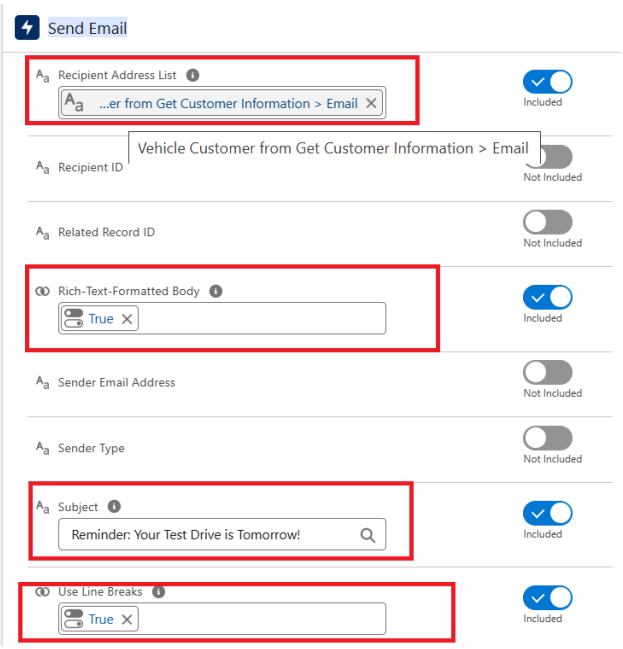


Figure 16: Send Email Action Configuration

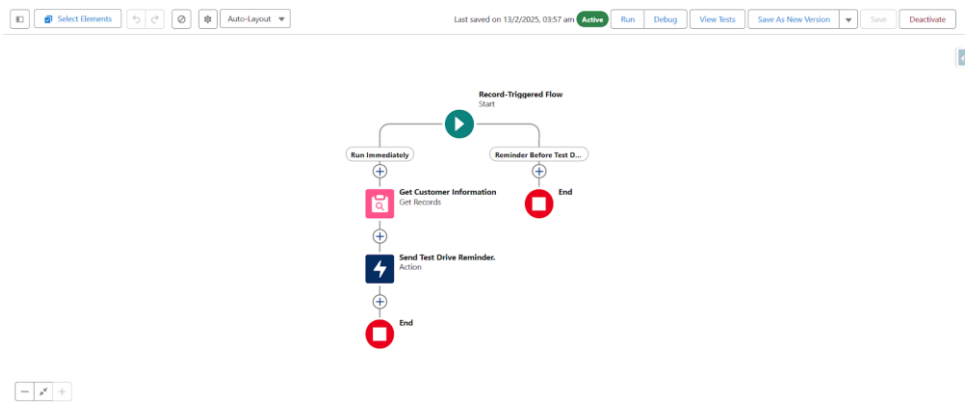


Figure 17: Test Drive Reminder Flow Canvas (Active)

The e-mail was delivered to the customer's mailbox (Figure 18). The received message begins "Dear test KHIT 123434" and states that the test drive for the record identifier shown is scheduled for tomorrow, signed "WhatNext Vision Motors Team".

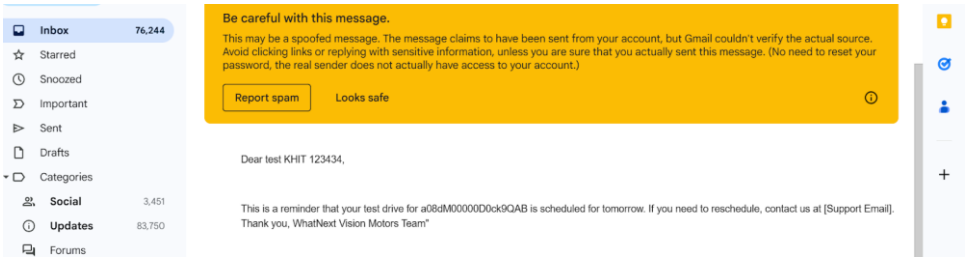


Figure 18: Test Drive Reminder E-mail Received

3.8.1 Observations on the implementation

- In the flow canvas (Figure 17), the Get Customer Information and Send Test Drive Reminder elements are placed on the **Run Immediately** path, while the **Reminder Before Test Drive** scheduled path leads directly to End. The e-mail is therefore sent when the test drive is saved, not one day before the test date. The one-day-before behaviour that the subject line implies is not implemented in the supplied configuration and is not verified.
- The received e-mail shows the record identifier instead of a vehicle name and contains the unreplaced text "[Support Email]". The message body should be revised before production use.

3.9 Salesforce Apex Automation

3.9.1 Batch Apex: VehicleOrderBatch

VehicleOrderBatch implements Database.Batchable<sObject>. Its purpose, as stated in the project documentation, is to handle the case where a customer places an order for an out-of-stock vehicle: the order remains Pending and, after new stock is added, the batch job confirms it. The start method queries all Vehicle_Order__c records with Status__c equal to Pending. For each batch, the execute method loads the stock quantity of the related vehicles and, for every order whose vehicle has a stock quantity above zero, sets the order status to Confirmed and subtracts one from the vehicle's stock quantity. It then updates the orders and the vehicles. The finish method writes a debug message.

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {


    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
```

```
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) { vehicleIds.add(order.Vehicle__c); }
        }
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c
                                       WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
            for (Vehicle_Order__c order : orderList) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c > 0) {
                        order.Status__c = 'Confirmed';
                        vehicle.Stock_Quantity__c -= 1;
                        ordersToUpdate.add(order);
                        vehiclesToUpdate.add(vehicle);
                    }
                }
            }
            if (!ordersToUpdate.isEmpty()) { update ordersToUpdate; }
            if (!vehiclesToUpdate.isEmpty()) { update vehiclesToUpdate; }
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
    }
}
```

Listing 3: VehicleOrderBatch (transcribed from the Developer Console screenshot)

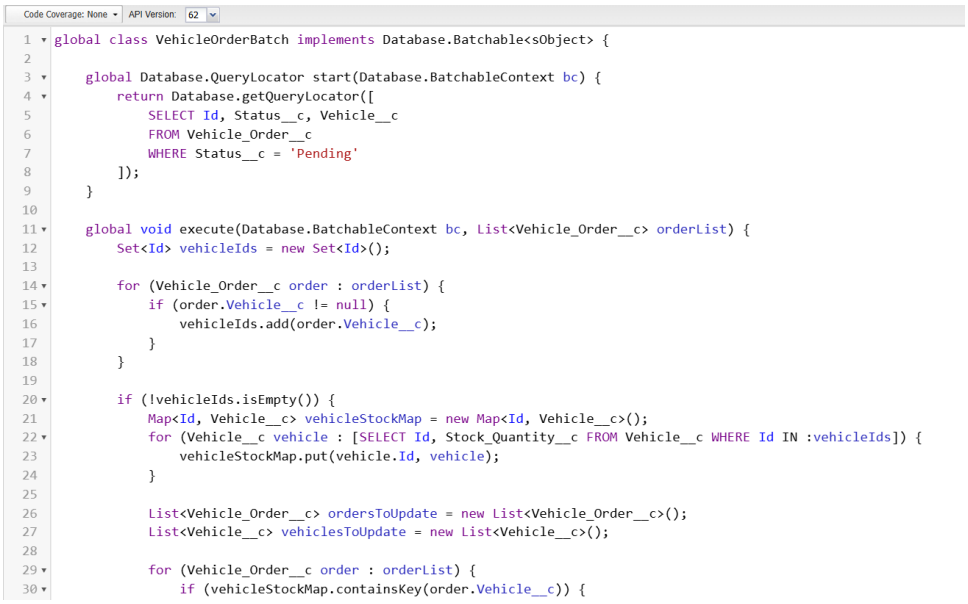


Figure 19: VehicleOrderBatch Class in the Developer Console

3.9.2 Scheduler: VehicleOrderBatchScheduler

VehicleOrderBatchScheduler implements Schedulable. When it executes, it creates an instance of VehicleOrderBatch and runs it with a batch size of 50.

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}
```

Listing 4: VehicleOrderBatchScheduler

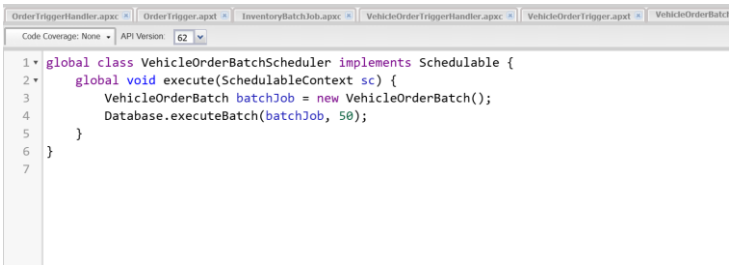


Figure 20: VehicleOrderBatchScheduler Class

3.9.3 Scheduling the batch job

The job is scheduled by running the following statements in the Execute Anonymous window of the Developer Console (Figure 21).

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
System.schedule('Daily Vehicle Order Processing', cronExp, new
        VehicleOrderBatchScheduler());
```

Listing 5: Scheduling statements

The project documentation describes the job as running "every night at midnight". The cron expression 0 0 12 * * ? means second 0, minute 0, hour 12, every day, which is **12:00 noon**, and the comment in the code and the Scheduled Jobs page both indicate 12:00 PM. This report therefore states the schedule as daily at 12:00 PM. The time zone applied is that of the Salesforce organisation, which was not documented.

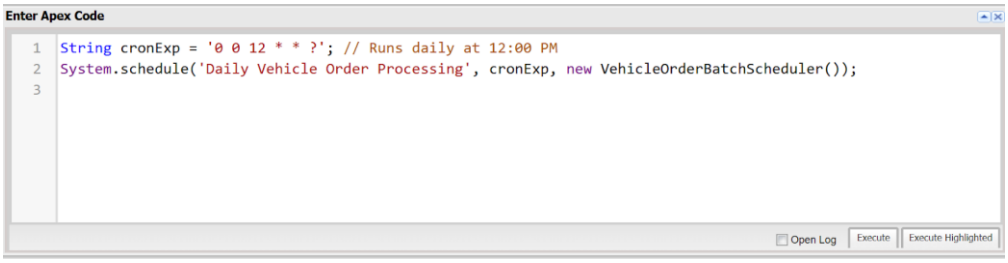


Figure 21: Scheduling the Batch Job in the Execute Anonymous Window

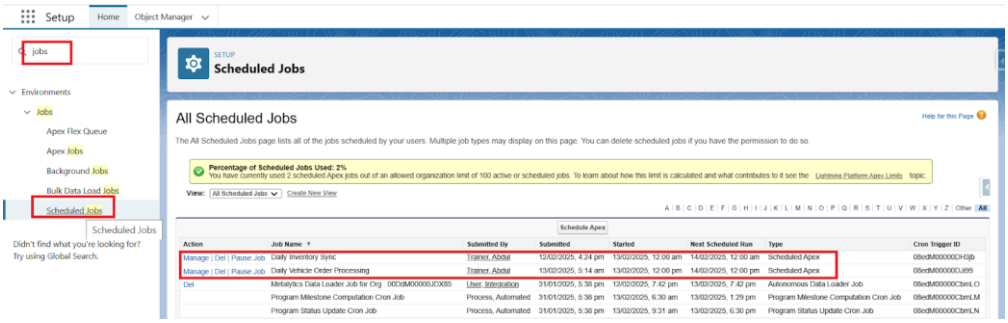


Figure 22: Scheduled Jobs Page Showing Daily Vehicle Order Processing

Figure 22 shows the job Daily Vehicle Order Processing, submitted on 13/02/2025, with the type Scheduled Apex and the next scheduled run at 12:00 pm on 14/02/2025. The same page lists a second scheduled Apex job, Daily Inventory Sync, scheduled for 12:00 am; the code behind that job was not supplied and it is not documented in this report.

3.9.4 Execution sequence

Table 16: Execution Sequence for a Vehicle Order

Step	Event
1	A user saves a Vehicle Order. The before-save trigger checks the stock quantity of the selected vehicle and rejects the save when it is zero or less
2	If the save succeeds, the after-save handler reduces stock by one when the status is Confirmed
3	For a newly created order with status Pending, the Auto Assign Dealer flow runs, retrieves the customer and looks up a dealer by exact location match
4	Every day at 12:00 PM the scheduled job runs VehicleOrderBatch, which confirms Pending orders whose vehicles have stock and reduces stock

3.10 Salesforce Security

The only security-related configuration evidenced in the project files is the assignment of the **System Administrator** profile to the WhatNext Vision Motors Lightning app (Figure 8), and the Field-Level Security step shown during creation of the Dealer lookup field, in which the field is made visible to profiles by default. No permission sets, roles, object permissions, field-level security restrictions, sharing rules or record-access restrictions for vehicle and order data were supplied. These aspects are therefore not verified, and the report makes no claim that they are configured. Securing vehicle and order data through profiles, permission sets and sharing settings is listed as a future enhancement.

3.11 Agentforce and AI Functionality

No Agentforce agent, Einstein AI feature or prompt template appears in the supplied project files or screenshots. AI functionality is therefore not implemented in this project as evidenced. AI-based demand forecasting appears in the project documentation only as future scope.

3.12 Reports and Dashboards

Reports and dashboards are listed in the project documentation as a functional requirement and as navigation items of the app, and the document refers to dashboard analytics in its testing section. No report definition, dashboard or screenshot was supplied, so this functionality is treated as planned and not verified.

3.13 Implementation Limitations and Concerns

A review of the supplied code and configuration identifies the following concerns. They were identified by code inspection and, except where stated, were not reproduced by testing.

- **Possible duplicate stock deduction.** The handler reduces stock whenever an order is saved with the status Confirmed, and the batch class also reduces stock when it sets an order to Confirmed. Because the batch class updates the order, the after-update trigger fires and the handler can deduct a second unit for the same order. In addition, any later edit of an already Confirmed order fires the after-update trigger again and can deduct stock again, because the handler does not compare the old and new status.
- **Duplicate records in the update list.** Both the handler and the batch class add the same Vehicle__c record to the update list once per order. When more than one order for the same vehicle is processed in one transaction, the list can contain the same record more than once, which Salesforce rejects with a duplicate-ID error. Stock changes should be aggregated per vehicle before the update.
- **Stock check applies to all statuses.** The before-save check rejects any insert or update when the vehicle stock is zero or less, so a Confirmed order for the last unit cannot later be edited (for example, set to Delivered) once stock reaches zero.

- **Dealer assignment.** The lookup is an exact text match, it is labelled "nearest" but does not calculate distance, and the evidence does not show the dealer being stored on the order (Section 3.7).
- **Reminder timing.** The Test Drive Reminder e-mail is sent on the immediate path rather than on the scheduled path one day before the test drive (Section 3.8).
- **Missing tests.** No Apex test classes were supplied; code coverage for deployment to production was not demonstrated.
- **Schedule statement.** The project documentation says midnight but the job runs at 12:00 PM.

CHAPTER 4: TESTING AND RESULTS

4.1 Testing Approach

Testing was carried out manually in the Salesforce organisation by performing actions in the user interface and observing the outcome, supported by screenshots. The areas considered were object and record creation, stock validation, order creation and status updates, stock deduction, customer and dealer lookup, flow execution, batch and scheduled jobs, and the user interface. Because the supplied evidence does not cover every scenario, each test case is recorded as Passed only where a screenshot or the code supports the stated result, and as Not verified otherwise.

4.2 Test Cases

Table 17: Functional Test Cases

ID	Scenario	Input / precondition	Expected result	Actual result (evidence)	Status
TC-01	Developer organisation setup	Account created and verified	Setup home page opens	Setup Home page displayed (Figure 1)	Passed
TC-02	Custom object creation	Create Vehicle object in Object Manager	Object listed as a custom object	Vehicle__c listed as Custom Object; other objects not shown	Partly passed (Vehicle only)
TC-03	Out-of-stock validation	Save order VC-0003, status Pending, for a vehicle with zero stock	Save rejected with out-of-stock error	Error "This vehicle is out of stock. Order cannot be placed." (Figure 23)	Passed
TC-04	Order creation with stock available	Vehicle with stock above zero; save new order	Order saved	No screenshot of a successful save	Not verified
TC-05	Order status update and stock deduction	Set order status to Confirmed	Vehicle stock reduced by 1	Code supports this; no before/after stock values shown	Not verified
TC-06	Repeated deduction	Edit a Confirmed order again; batch confirms a Pending order	Stock reduced once per order	Code review indicates possible repeated deduction (Section 3.13); not run	Not verified

ID	Scenario	Input / precondition	Expected result	Actual result (evidence)	Status
TC-07	Customer lookup in flow	Create Pending order with a customer	Customer record retrieved	Flow configured (Figure 11); execution not shown	Not verified
TC-08	Dealer lookup by location	Customer address equals a dealer location	Matching dealer retrieved and assigned	Flow configured (Figures 12, 13); no result shown	Not verified
TC-09	Test-drive reminder e-mail	Save test drive with status Scheduled	E-mail sent to customer	E-mail received (Figure 18)	Passed
TC-10	Reminder sent one day before test date	Test drive date set in the future	E-mail sent one day before date	E-mail action is on the immediate path; scheduled path is empty (Figure 17)	Not met / Not verified
TC-11	Batch confirms pending orders	Pending order, vehicle stock above zero; run batch	Order Confirmed; stock reduced	Code only; no run output	Not verified
TC-12	Scheduled job registration	Run scheduling statements	Job listed on Scheduled Jobs page	Daily Vehicle Order Processing listed; next run 14/02/2025, 12:00 pm (Figure 22)	Passed
TC-13	Reports and dashboards	Open reports and dashboard	Order and dealer metrics shown	No evidence supplied	Not verified
TC-14	Access control	Log in with non-administrator user	Restricted access to vehicle and order data	No evidence supplied	Not verified

4.3 Results and Observations

4.3.1 Stock validation

The stock validation test produced the expected behaviour. Figure 23 shows the Vehicle Order record VC-0003 with the customer "test KHIT 123434", the vehicle "1erwe", the order date 21/02/2025 and the status Pending; saving it displays the error "This vehicle is out of stock. Order cannot be placed." and the record is not saved.

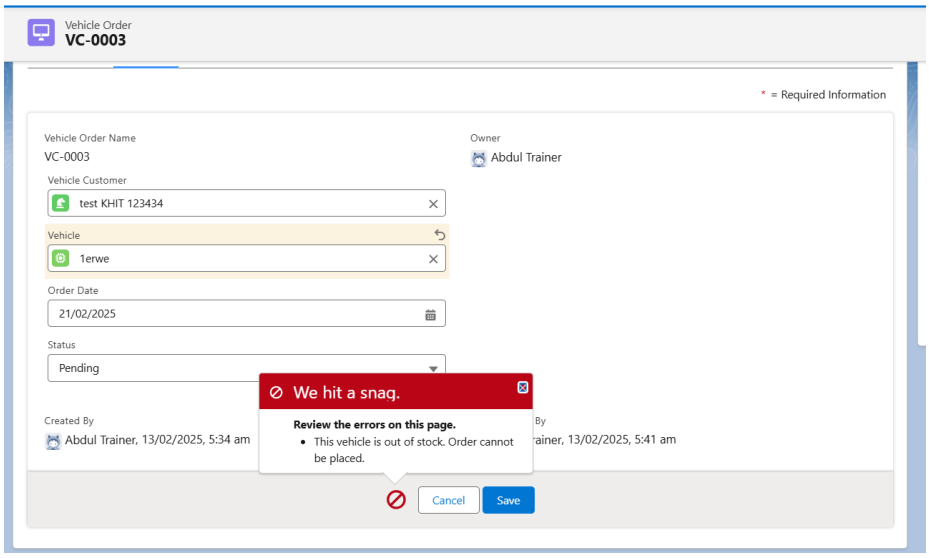


Figure 23: Vehicle Order Form Showing the Out-of-Stock Validation Error

4.3.2 Test-drive reminder

The reminder e-mail was delivered (Figure 18), confirming that the Get Records and Send Email elements work with the customer's e-mail address. The timing and content observations of Section 3.8 remain open.

4.3.3 Scheduled processing

The scheduled job is registered and queued (Figure 22). The supplied evidence does not include a job execution log or the changes made to orders and stock by a completed run.

4.3.4 User interface validation

The Lightning record form for Vehicle Order displays the fields and the validation message as expected (Figure 23). Screenshots of the list views and record pages of the other objects, and of the app home page, were not supplied.

4.4 Summary of Implementation Status

Table 18: Implementation Status Summary

Feature	Evidence	Status
Custom objects and fields	Specification; creation of Vehicle, Vehicle Model, Stock Quantity and Dealer lookup shown	Partially verified
Tabs and Lightning app	Creation steps and profile assignment shown; finished app not shown	Partially verified
Stock validation	Trigger and handler code; error message screenshot	Verified

Feature	Evidence	Status
Stock deduction on Confirmed	Handler code only	Not verified
Dealer selection by location	Flow configuration; exact match; no result shown	Partially implemented, not verified
Test-drive reminder e-mail	Flow configuration; e-mail received; timing differs from design	Partially verified
Batch processing of pending orders	Code only	Not verified
Daily scheduling	Scheduled Jobs page	Verified
Reports and dashboards	None	Not verified
Security configuration	Profile assignment to app only	Not verified
Agentforce / AI	None	Not implemented

CHAPTER 5: CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Project Summary

WhatNext Vision Motors is a Salesforce Lightning application for vehicle and dealership management. It provides a data model of six custom objects, a custom Lightning app, a before-save and after-save Apex trigger with a handler class for stock validation and stock update, a record-triggered flow for dealer lookup, a record-triggered flow for test-drive reminders, and a batch Apex job scheduled daily at 12:00 PM.

5.2 Objectives Achieved

Table 19: Objectives and Status

Objective	Status
Centralised data model for vehicles, dealers, customers, orders, test drives and service requests	Achieved in design; partially evidenced by screenshots
Prevent orders for out-of-stock vehicles	Achieved and verified
Keep stock consistent with confirmed orders	Implemented in code; not verified, with risk of duplicate deduction
Associate customers with dealers by location	Partially achieved; exact text match, result not shown
E-mail reminders for test drives	Partially achieved; e-mail delivered, one-day-before timing not implemented
Daily batch processing of pending orders	Scheduled and verified as registered; execution results not shown

5.3 Advantages

- All vehicle, dealer, customer and order data is held in one cloud platform with a single source of truth.
- Stock validation is enforced automatically at the moment an order is saved.
- Declarative flows and Apex code automate dealer lookup, reminders and batch processing with little manual effort.
- The Lightning user interface and the cloud platform give browser-based access from anywhere.

- The data model and automation can be extended to additional dealers and objects without redesign.

5.4 Current Limitations

- Possible duplicate stock deduction and duplicate-ID update errors (Section 3.13).
- Dealer selection uses exact text matching; it is not a distance-based nearest-dealer calculation, and the dealer is not shown to be stored on the order.
- The test-drive reminder is sent immediately instead of one day before the test drive, and its message text contains an unreplaced placeholder.
- Reports, dashboards, detailed security configuration and AI functionality are not evidenced.
- No Apex test classes were supplied; many test cases remain unverified.
- The application depends on the Salesforce platform and requires initial configuration effort.

5.5 Future Enhancements

The following items are proposed enhancements and are **not** part of the current implementation.

- **Distance-based dealer recommendation:** store geolocation for customers and dealers and select the dealer with the smallest distance, with the dealer saved in a lookup field on the order.
- **Improved stock transaction handling:** aggregate stock changes per vehicle, deduct stock only when the status changes to Confirmed, and prevent the batch class and trigger from deducting twice.
- **Stronger validation:** apply the stock check only to orders that are Pending or Confirmed, and add validation rules for mandatory fields.
- **Notifications:** move the e-mail action to the scheduled path, use an e-mail template with the vehicle name and a support address, and add SMS notifications.
- **Reporting dashboards:** build reports and dashboards showing order status, dealer performance and sales metrics.

- **Additional security controls:** define profiles, permission sets, field-level security and sharing rules for vehicle and order data.
- **Apex unit tests:** write test classes with adequate coverage for the trigger, handler, batch and scheduler.
- **Other ideas from the project documentation:** a mobile ordering app, dealer GPS integration, and AI-based demand forecasting.

5.6 Conclusion

The project demonstrates how Salesforce can be used to replace manual registers and spreadsheets with a connected vehicle ordering system. The stock validation, test-drive e-mail and scheduled job are supported by evidence, while the remaining automation is implemented but not yet verified, and several concerns have been identified that should be resolved before production use. With the proposed enhancements, the application can evolve into a dependable platform for dealership operations.

References

[1] Salesforce Developers, "Apex Developer Guide", https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/

[2] Salesforce, "Salesforce Help: Flow Builder and Record-Triggered Flows", https://help.salesforce.com/

[3] Salesforce Developers, "Object Reference for the Salesforce Platform", https://developer.salesforce.com/docs/

[4] Salesforce Trailhead, "Apex Basics and Database", https://trailhead.salesforce.com/

[5] WhatNext Vision Motors project documentation and Salesforce screenshots supplied by the project team.

Thank you…!!!